

Student Management System

Full Stack Development Project

HTML / CSS / JavaScript + Node.js (Express) + MySQL

Submitted by: _____

Roll No: _____ Course: _____

1. Project Overview

The Student Management System is a full stack web application used to store and manage student records. A user can add a new student, view all students in a table, search by name or course, edit details and delete a record. The project follows a three-tier architecture: a browser-based frontend, a Node.js REST API and a MySQL database.

Features

- Add, view, update and delete students (full CRUD)
- Live search by name or course
- Validation on both frontend and backend
- Duplicate email detection
- SQL injection protection using parameterised queries
- XSS protection by escaping output in the frontend
- Responsive layout that works on mobile and desktop

Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	User interface and API calls (fetch)
Backend	Node.js, Express.js	REST API and static file server
Database	MySQL (mysql2 driver)	Persistent storage of student records

Folder Structure

```
student-management-system/  
|-- package.json  
|-- server.js      <- Node.js + Express backend (REST API)  
|-- schema.sql     <- MySQL database and table  
`-- public/  
    |-- index.html  <- Frontend page  
    |-- style.css   <- Styling  
    `-- script.js   <- Frontend logic (fetch API calls)
```

2. Database Design

A single table, **students**, stores all records. The email column is **UNIQUE** so the same email cannot be registered twice.

Column	Type	Constraint
id	INT	Primary key, auto increment
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
course	VARCHAR(100)	NOT NULL
year	INT	NOT NULL
phone	VARCHAR(15)	Optional
created_at	TIMESTAMP	Default current time

schema.sql

```
CREATE DATABASE IF NOT EXISTS student_db;
USE student_db;

CREATE TABLE IF NOT EXISTS students (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(100) NOT NULL UNIQUE,
  course      VARCHAR(100) NOT NULL,
  year        INT NOT NULL,
  phone       VARCHAR(15),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (name, email, course, year, phone) VALUES
('Aarav Sharma', 'aarav@example.com', 'B.Tech CSE', 2, '9876543210'),
('Priya Singh', 'priya@example.com', 'BCA', 1, '9123456780');
```

3. Backend (Node.js + Express)

The server exposes a REST API under **/api/students** and also serves the frontend files from the **public** folder.

API Endpoints

Method	Endpoint	Description
GET	/api/students?search=	List students (optional search)
POST	/api/students	Add a new student
PUT	/api/students/:id	Update a student
DELETE	/api/students/:id	Delete a student

package.json

```
{
  "name": "student-management-system",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  }
}
```

```

    },
    "dependencies": {
      "cors": "^2.8.5",
      "express": "^4.19.2",
      "mysql2": "^3.9.7"
    }
  }
}

```

server.js

```

const express = require('express');
const cors = require('cors');
const mysql = require('mysql2');
const path = require('path');

const app = express();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// ---- MySQL connection pool (change password to your own) ----
const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  database: 'student_db'
});

// Common error handler
function fail(res, err) {
  if (err.code === 'ER_DUP_ENTRY')
    return res.status(409).json({ error: 'Email already exists' });
  res.status(500).json({ error: err.message });
}

// READ: list all students (optional ?search=)
app.get('/api/students', (req, res) => {
  const q = '%' + (req.query.search || '') + '%';
  db.query(
    'SELECT * FROM students WHERE name LIKE ? OR course LIKE ? ORDER BY id DESC',
    [q, q],
    (err, rows) => (err ? fail(res, err) : res.json(rows))
  );
});

// CREATE: add a student
app.post('/api/students', (req, res) => {
  const { name, email, course, year, phone } = req.body;
  if (!name || !email || !course || !year)
    return res.status(400).json({ error: 'Required fields missing' });
  db.query(
    'INSERT INTO students (name, email, course, year, phone) VALUES (?, ?, ?, ?, ?)',
    [name, email, course, year, phone || null],
    (err, result) =>
      err ? fail(res, err) : res.status(201).json({ id: result.insertId })
  );
});

// UPDATE: edit a student
app.put('/api/students/:id', (req, res) => {
  const { name, email, course, year, phone } = req.body;
  db.query(
    'UPDATE students SET name=?, email=?, course=?, year=?, phone=? WHERE id=?',
    [name, email, course, year, phone || null, req.params.id],
    (err, result) => {
      if (err) return fail(res, err);
      if (result.affectedRows === 0)
        return res.status(404).json({ error: 'Student not found' });
      res.json({ message: 'Updated' });
    }
  );
});

// DELETE: remove a student
app.delete('/api/students/:id', (req, res) => {

```

```
db.query('DELETE FROM students WHERE id=?', [req.params.id], (err, result) => {
  if (err) return fail(res, err);
  if (result.affectedRows === 0)
    return res.status(404).json({ error: 'Student not found' });
  res.json({ message: 'Deleted' });
});
});

const PORT = 3000;
app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

4. Frontend (HTML, CSS, JavaScript)

The frontend is a single page. It calls the REST API with **fetch()** and updates the table without reloading the page.

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Management System</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header><h1>Student Management System</h1></header>

  <main>
    <section class="card">
      <h2 id="formTitle">Add Student</h2>
      <form id="studentForm">
        <input name="name" placeholder="Full name" required>
        <input name="email" type="email" placeholder="Email" required>
        <input name="course" placeholder="Course (e.g. B.Tech CSE)" required>
        <input name="year" type="number" min="1" max="6" placeholder="Year" required>
        <input name="phone" placeholder="Phone">
        <div class="actions">
          <button type="submit">Save</button>
          <button type="button" id="cancelBtn" class="secondary" hidden>Cancel</button>
        </div>
      </form>
    </section>

    <section class="card">
      <input id="search" placeholder="Search by name or course...">
      <div class="table-wrap">
        <table>
          <thead>
            <tr>
              <th>ID</th><th>Name</th><th>Email</th><th>Course</th>
              <th>Year</th><th>Phone</th><th>Actions</th>
            </tr>
          </thead>
          <tbody id="tbody"></tbody>
        </table>
      </div>
    </section>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

public/style.css

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f3f5f9;
  color: #222;
}

header {
  background: #2f4ea2;
  color: #fff;
  padding: 16px 24px;
}

header h1 { margin: 0; font-size: 22px; }

main { max-width: 960px; margin: 24px auto; padding: 0 16px; }
```

```

.card {
  background: #fff;
  border-radius: 8px;
  padding: 18px;
  margin-bottom: 20px;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.1);
}
.card h2 { margin-top: 0; font-size: 18px; }

form {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 10px;
}

input {
  padding: 9px 10px;
  border: 1px solid #c5cad6;
  border-radius: 6px;
  font-size: 14px;
}
#search { width: 100%; margin-bottom: 12px; }

button {
  padding: 9px 16px;
  border: none;
  border-radius: 6px;
  background: #2f4ea2;
  color: #fff;
  cursor: pointer;
  font-size: 14px;
}
button.secondary { background: #6b7280; }
button.danger { background: #d64545; }
button:hover { opacity: 0.9; }

.actions { display: flex; gap: 8px; }

.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; }
th, td {
  padding: 10px;
  text-align: left;
  border-bottom: 1px solid #e5e7eb;
  font-size: 14px;
}
th { background: #eef1f8; }
td button { padding: 5px 10px; font-size: 12px; margin-right: 4px; }

```

public/script.js

```

const API = '/api/students';
const form = document.getElementById('studentForm');
const tbody = document.getElementById('tbody');
const searchBox = document.getElementById('search');
const formTitle = document.getElementById('formTitle');
const cancelBtn = document.getElementById('cancelBtn');

let students = [];
let editId = null;

// Escape text before putting it into HTML (prevents XSS)
const esc = (s) => String(s ?? '').replace(/[<>"']/g, (c) => (
  { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]
));

async function loadStudents() {
  const res = await fetch(API + '?search=' + encodeURIComponent(searchBox.value));
  students = await res.json();
  tbody.innerHTML = students.map((s) => `
    <tr>
      <td>${s.id}</td><td>${esc(s.name)}</td><td>${esc(s.email)}</td>
      <td>${esc(s.course)}</td><td>${s.year}</td><td>${esc(s.phone)}</td>
    <td>
      <button onclick="editStudent(${s.id})">Edit</button>
    </td>
  `);
}

```

```

        <button class="danger" onclick="deleteStudent(${s.id})">Delete</button>
      </td>
    </tr>` ).join('');
  }

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const body = Object.fromEntries(new FormData(form));
    const res = await fetch(editId ? `${API}/${editId}` : API, {
      method: editId ? 'PUT' : 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
    if (!res.ok) {
      const data = await res.json();
      alert(data.error || 'Something went wrong');
      return;
    }
    resetForm();
    loadStudents();
  });

  function editStudent(id) {
    const s = students.find((x) => x.id === id);
    for (const key of ['name', 'email', 'course', 'year', 'phone']) {
      form.elements[key].value = s[key] ?? '';
    }
    editId = id;
    formTitle.textContent = 'Edit Student';
    cancelBtn.hidden = false;
    window.scrollTo({ top: 0, behavior: 'smooth' });
  }

  async function deleteStudent(id) {
    if (!confirm('Delete this student?')) return;
    await fetch(`${API}/${id}`, { method: 'DELETE' });
    loadStudents();
  }

  function resetForm() {
    form.reset();
    editId = null;
    formTitle.textContent = 'Add Student';
    cancelBtn.hidden = true;
  }

  cancelBtn.addEventListener('click', resetForm);
  searchBox.addEventListener('input', loadStudents);

  loadStudents();

```

5. Output Screenshots

Student Management System

Add Student

ID	Name	Email	Course	Year	Phone	Actions
2	Priya Singh	priya@example.com	BCA	1	9123456780	Edit Delete
1	Aarav Sharma	aarav@example.com	B.Tech CSE	2	9876543210	Edit Delete

Fig 1: Home page with the student list

Student Management System

Add Student

ID	Name	Email	Course	Year	Phone	Actions
2	Priya Singh	priya@example.com	BCA	1	9123456780	Edit Delete
1	Aarav Sharma	aarav@example.com	B.Tech CSE	2	9876543210	Edit Delete

Fig 2: Add Student form filled with new details

Student Management System

Add Student

ID	Name	Email	Course	Year	Phone	Actions
3	Rohan Verma	rohan@example.com	B.Sc IT	3	9988776655	Edit Delete
2	Priya Singh	priya@example.com	BCA	1	9123456780	Edit Delete
1	Aarav Sharma	aarav@example.com	B.Tech CSE	2	9876543210	Edit Delete

Fig 3: New student added to the table

Student Management System

Edit Student

ID	Name	Email	Course	Year	Phone	Actions
3	Rohan Verma	rohan@example.com	B.Sc IT	3	9988776655	Edit Delete
2	Priya Singh	priya@example.com	BCA	1	9123456780	Edit Delete
1	Aarav Sharma	aarav@example.com	B.Tech CSE	2	9876543210	Edit Delete

Fig 4: Edit mode with Save and Cancel buttons

Student Management System

Add Student

Full name

Email

Course (e.g. B.Tech CSE)

Year

Phone

Save

BCA

ID	Name	Email	Course	Year	Phone	Actions
2	Priya Singh	priya@example.com	BCA	1	9123456780	EditDelete

Fig 5: Search filters the list by name or course

Student Management System

Add Student

Full name

Email

Course (e.g. B.Tech CSE)

Year

Phone

Save

Search by name or course...

ID	Name	Email	Course	Year	Phone	Actions
3	Rohan Verma	rohan@example.com	B.Sc Computer Science	3	9988776655	EditDelete
2	Priya Singh	priya@example.com	BCA	1	9123456780	EditDelete

Fig 6: Table after deleting a student

Student Management System - Full Stack Project

Page 11

6. How to Run the Project

1. Install Node.js (v18+) and MySQL Server.
2. Create the project folder and add all files as shown in this document.
3. Create the database and table:
`mysql -u root -p < schema.sql`
4. Open server.js and set your MySQL password in the connection pool.
5. Install dependencies:
`npm install`
6. Start the server:
`npm start`
7. Open the browser at:
`http://localhost:3000`

7. Future Improvements

- Admin login with JWT authentication
- Attendance and marks modules
- Pagination and export to CSV/PDF
- Deployment on a cloud server

8. Conclusion

This project demonstrates a complete full stack workflow: designing a relational database, building a REST API with Node.js and Express, and creating an interactive frontend with plain HTML, CSS and JavaScript.